

UNIDAD 5 · SEMANA 7

Tuberías y composición de procesos

📅 Semana 7 · fechas por confirmar </> Laboratorio TD4 (resuelto + desafío IA)

📺 2 animaciones

LABORATORIO TD4 · RESUELTO

Tuberías sin nombre y composición de procesos

Cómo construye el shell `ls | grep | wc`: una tubería por cada `|`, un proceso por comando, `dup2` para conectar la salida de uno con la entrada del siguiente y, sobre todo, cerrar a tiempo los extremos que no se usan.

✓ Informe del laboratorio

Entrega este laboratorio con un informe corto en LaTeX, con la plantilla del curso: cada ejercicio explicado, el código que importa y lo que ejecutaste en la terminal. Puedes usar IA generativa para escribir el LaTeX: decláralo y verifica cada resultado. Plantilla, ejemplo y guía de Prism →

EJERCICIOS

- | | | | |
|---|--|---|---------------------------------------|
| 1 | Una tubería entre padre e hijo | 4 | Una tubería de comandos fija |
| 2 | Las reglas de las tuberías: capacidad, bloqueo y SIGPIPE | 5 | La tubería desde la línea de comandos |
| 3 | <code>ps</code> en mayúsculas | | |

TD4 · 1

● Resuelto · probado

Una tubería entre padre e hijo

Enunciado. Crea una tubería con `pipe(2)` y duplica el proceso. El padre escribe 10 veces "Hello World", avisa por la salida de error que se va y termina. El hijo lee como máximo 500 bytes de una sola vez, los muestra, avisa y termina. Usa macros para las constantes.

Solución



C tds/td4/mypipe.c

```

/*
 * TD4 - exercises 1 and 2: an unnamed pipe between a parent (writer) and a
 * child (reader).
 * ./mypipe [count]      the parent writes "Hello World\n" `count` times
 * (default 10)
 * The child reads AT MOST 500 bytes, once, prints them and leaves.
 *
 * Exercise 2: with a large count (10000 x 12 bytes = 120 KB > 64 KiB, the
 * pipe capacity)
 * the parent blocks on write() once the pipe is full. When the child exits,
 * nobody can read
 * any more: the next write() raises SIGPIPE and kills the parent ("Broken
 * pipe", status 141
 * in the shell). That only works if the parent CLOSED its own read end: with
 * -DKEEP_READ_END (Makefile target mypipe_bug) the parent itself is still a
 * potential reader,
 * the kernel never raises SIGPIPE and the parent blocks forever.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define MESSAGE  "Hello World\n"
#define READ_MAX 500

int main(int argc, char *argv[])
{
    int count = argc > 1 ? atoi(argv[1]) : 10;
    int fds[2];                                /* fds[0] read end,
    fds[1] write end */

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                             /* child: reader */
        exit_if(close(fds[1]) == -1, "close"); /* never writes */
        char buffer[READ_MAX];
        ssize_t n = read(fds[0], buffer, READ_MAX); /* one single read */
    }

```

```

    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
    fprintf(stderr, "I am the child, I read %zd bytes and I leave\n", n);
    exit_if(close(fds[0]) == -1, "close");
    return EXIT_SUCCESS;
}

/* parent: writer */
#ifdef KEEP_READ_END
    exit_if(close(fds[0]) == -1, "close");          /* THE line that
exercise 2 is about */
#endif
    for (int i = 0; i < count; i++)
        exit_if(write_all(fds[1], MESSAGE, strlen(MESSAGE)) == -1, "write");
    fprintf(stderr, "I am the parent, I wrote %d messages and I leave\n",
count);
    exit_if(close(fds[1]) == -1, "close");
    exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");
    return EXIT_SUCCESS;
}

```

Salida real

```

$ ./mypipe
I am the parent, I wrote 10 messages and I leave
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
I am the child, I read 120 bytes and I leave

```

`fds[0]` es el extremo de lectura y `fds[1]` el de escritura (como 0 = entrada y 1 = salida). Después de `fork` los dos procesos tienen los dos extremos: cada uno cierra el que no usa.

Las reglas de las tuberías: capacidad, bloqueo y SIGPIPE

Enunciado. Pasa como argumento cuántas veces escribe el padre: 50, 100, 500, 1000, 5000, 10000... Si el programa parece bloquearse, revisa el estado de los procesos, dibuja las tablas de descriptores y corrige. Con el programa corregido, ¿cuál es su estado de terminación?

Solución

Salida real

```
$ python3 -c 'import os, fcntl; r, w = os.pipe(); print(fcntl.fcntl(w,
fcntl.F_GETPIPE_SZ), "bytes")'
65536 bytes
```

Salida real

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $?" (124 =
killed by timeout: it was blocked)"
I am the child, I read 500 bytes and I leave
exit status: 124 (124 = killed by timeout: it was blocked)
```

Salida real

```
$ ./mypipe 10000 > /dev/null; echo "exit status of mypipe: $?"
I am the child, I read 500 bytes and I leave
exit status of mypipe: 141
```

Nuevo: la carrera, medida. El enunciado original sugería buscar "a partir de cuántos mensajes" el padre muere. En un equipo con varios núcleos no hay un umbral fijo: el hijo lee 500 bytes y sale mientras el padre sigue escribiendo, y quién gana depende del planificador. 30 ejecuciones por tamaño:

Salida real

```
$ for n in 50 100 ... 1000; do run ./mypipe n 30 times; count exit statuses;
done
  50 messages: 30 x status 0      0 x status 141 (SIGPIPE)
 100 messages: 30 x status 0      0 x status 141 (SIGPIPE)
 200 messages: 28 x status 0      2 x status 141 (SIGPIPE)
 300 messages: 19 x status 0     11 x status 141 (SIGPIPE)
 400 messages:  5 x status 0     25 x status 141 (SIGPIPE)
 500 messages:  5 x status 0     25 x status 141 (SIGPIPE)
1000 messages:  0 x status 0     30 x status 141 (SIGPIPE)
```

Míralo animado → A5.2

1. La tubería tiene capacidad limitada: **65 536 bytes** por defecto en Linux. $10\,000 \times 12$ bytes = 120 000 bytes no caben: cuando se llena, `write` *bloquea* al padre hasta que alguien lea.
2. El hijo lee 500 bytes y termina. Si el padre **no cerró** su propio extremo de lectura (`mypipe_bug`), el núcleo ve que todavía existe un lector —el propio padre— y lo deja bloqueado para siempre: `timeout` lo mata a los 3 s (estado 124).
3. Con el extremo cerrado, al irse el hijo ya no hay lectores: el siguiente `write` recibe la señal **SIGPIPE**, cuya acción por defecto es terminar el proceso. El shell muestra "Broken pipe" y el estado $141 = 128 + 13$.

Animación A5.2 · Capacidad de la tubería →

TD4 · 3

● Resuelto · probado

ps en mayúsculas

Enunciado. El hijo lee continuamente de la tubería y muestra lo leído en mayúsculas. El padre redirige su salida estándar hacia la tubería con `dup2` y ejecuta `ps` con `execlp`.

Solución

```
C tds/td4/upper.c
```

```
/*
 * TD4 - exercise 3: ps | uppercase, built by hand.
```

```

* The child reads the pipe until end of file and prints it in uppercase.
* The parent redirects its stdout to the pipe (dup2) and becomes `ps`
(execlp).
* The child sees end of file when `ps` exits, because it was the last
writer.
*/
#include <ctype.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define BUFFER_SIZE 512

static void to_upper(int fd_in, int fd_out)
{
    char buffer[BUFFER_SIZE];
    ssize_t n;

    while ((n = read(fd_in, buffer, sizeof buffer)) > 0) {    /* until end
of file */
        for (ssize_t i = 0; i < n; i++)
            buffer[i] = (char)toupper((unsigned char)buffer[i]);
        exit_if(write_all(fd_out, buffer, (size_t)n) == -1, "write");
    }
    exit_if(n == -1, "read");
}

int main(void)
{
    int fds[2];

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                                          /* child */
        exit_if(close(fds[1]) == -1, "close");             /* otherwise it never sees
end of file */
        to_upper(fds[0], STDOUT_FILENO);
        return EXIT_SUCCESS;
    }

    /* parent */
    exit_if(close(fds[0]) == -1, "close");

```

```
exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
exit_if(close(fds[1]) == -1, "close");      /* 1 already refers to the
pipe */
execlp("ps", "ps", (char *)NULL);          /* "ps" twice: file to run,
then argv[0] */
exit_if(1, "execlp ps");
return EXIT_FAILURE;
}
```

Salida real

```
$ ./upper | head -4
  PID TTY          TIME CMD
 6706 ?           00:00:25 UVICORN
 9771 ?           00:00:03 SYSTEMD
 9801 ?           00:00:00 (SD-PAM)
```

El hijo termina cuando `read` devuelve 0, y eso pasa solo cuando se cierran *todos* los extremos de escritura: el de `ps` (al terminar) y el suyo propio, que por eso cierra al empezar. `ps` no sabe que escribe en una tubería: su descriptor 1 simplemente apunta a ella.

TD4 · 4

● Resuelto · probado

Una tubería de comandos fija

Enunciado. Ejecuta la tubería definida en una variable `commands` (por ejemplo `ls -l | tr -d '[:blank:]'`), con cualquier número de comandos. El programa termina cuando termina el último y devuelve su estado; si murió por una señal, lo informa y devuelve `EXIT_FAILURE`.

Solución

C tds/td4/pipeline.c

```
/*
 * TD4 - exercise 4: run a fixed pipeline, like the shell does for ls -l |
tr -d '[:blank:]'
 * Works for any number of commands. Returns the exit status of the LAST
command, or
```

```

    * EXIT_FAILURE (with a message) if it was killed by a signal.
    */
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

static char *ls[] = { "ls", "-l", NULL };
static char *tr[] = { "tr", "-d", "[:blank:]", NULL };
static char **commands[] = { ls, tr };
static const int ncommands = sizeof commands / sizeof commands[0];

int main(void)
{
    int in = STDIN_FILENO;      /* where the next command reads from */
    pid_t last = -1;

    for (int i = 0; i < ncommands; i++) {
        int fds[2] = { -1, -1 };
        if (i < ncommands - 1) /* every command but
the last writes to a pipe */
            exit_if(pipe(fds) == -1, "pipe");

        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0) {
            if (in != STDIN_FILENO) { /* not the first: read
from previous pipe */
                exit_if(dup2(in, STDIN_FILENO) == -1, "dup2 stdin");
                exit_if(close(in) == -1, "close");
            }
            if (fds[1] != -1) { /* not the last: write
to the new pipe */
                exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2 stdout");
                exit_if(close(fds[0]) == -1 || close(fds[1]) == -1, "close");
            }
            execvp(commands[i][0], commands[i]);
            perror(commands[i][0]);
            _exit(127);
        }

        /* parent: close what the child inherited, keep only the read end for
the next one */
    }
}

```

```

    if (in != STDIN_FILENO)
        exit_if(close(in) == -1, "close");
    if (fds[1] != -1) {
        exit_if(close(fds[1]) == -1, "close");
        in = fds[0];
    }
    last = pid;
}

/* Wait for every child (no zombies), remember the status of the last
one. */
int status, last_status = 0;
pid_t pid;
while ((pid = wait(&status)) > 0)
    if (pid == last)
        last_status = status;

if (WIFEXITED(last_status))
    return WEXITSTATUS(last_status);
if (WIFSIGNALED(last_status))
    fprintf(stderr, "pipeline: last command killed by %s\n",
    signame(WTERMSIG(last_status)));
return EXIT_FAILURE;
}

```

Salida real

```

$ cd demo && ../pipeline
total8
-rw-rw-r--1studentstudent4Sep3013:13one.txt
-rw-rw-r--1studentstudent2Sep3013:13two.txt

```

El patrón: en cada vuelta se crea la tubería *siguiente* (salvo para el último comando); el hijo toma como entrada la tubería anterior (`in`) y como salida la nueva; el padre cierra lo que el hijo ya heredó y guarda el extremo de lectura para la siguiente vuelta. Al final, el padre espera a **todos** los hijos (si no, quedarían zombis) y se queda con el estado del último.

La tubería desde la línea de comandos

Enunciado. Igual que el anterior, pero los comandos llegan por la línea de comandos separados por `--`: `pipeline ls -l -- grep a -- wc -l` equivale a `ls -l | grep a | wc -l`.

Solución

C tds/td4/pipeline_args.c

```
int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s cmd [args] [-- cmd [args]]...\n",
argv[0]);
        return EXIT_FAILURE;
    }

    /* Split argv in place. commands[k] points to the first word of command
k. */
    char **commands[argc];
    int ncommands = 0;
    commands[ncommands++] = &argv[1];
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--") == 0) {
            argv[i] = NULL; /* ends the previous
command */
            if (i + 1 ≥ argc || strcmp(argv[i + 1], "--") == 0 ||
commands[ncommands - 1][0] == NULL) {
                fprintf(stderr, "%s: empty command\n", argv[0]);
                return EXIT_FAILURE;
            }
            commands[ncommands++] = &argv[i + 1];
        }
    }
}
```

Salida real

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

Salida real

```
$ ./pipeline_args ls -- sh -c 'exit 3'; echo "status: $?"
status: 3
```

Salida real

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

Truco: `argv` ya es un arreglo de cadenas terminado en `NULL`. Reemplazando cada `--` por `NULL`, cada comando pasa a ser un tramo de `argv` listo para `execvp`, sin copiar nada. El resto es idéntico al ejercicio 4.

Lista de cotejo

Desafío con IA generativa · Tuberías en tu mini-shell



Objetivo. Agregar el operador `|` al mini-shell reutilizando `pipeline_args.c`, y diagnosticar con la IA un interbloqueo provocado a propósito.

Prompt inicial. Úsalo en la herramienta que prefieras (Claude, ChatGPT, Gemini, Copilot, Codex...) con la versión *resuelta* del TD como contexto. Pide primero un plan y revisa cada paso antes de aceptarlo.

Prompt

Tengo un mini-shell y un programa que ejecuta tuberías de N comandos [pega ambos].
(1) Ayúdame a integrar el operador `|` en el shell. (2) Luego voy a quitar a propósito un `close()` en el padre: explícame, dibujando las tablas de descriptores de cada proceso, por qué ``ls | wc -l`` ya no termina, y cómo lo confirmarías con ``ls -l /proc/<pid>/fd``. No corrijas el código tú: guíame con preguntas.

Cómo verificar el resultado

- ``ls -l | grep c | wc -l`` da lo mismo que en bash.
- Con el `close()` eliminado el comando se queda bloqueado y ``/proc/<pid>/fd`` muestra el extremo de escritura abierto.
- Puedes nombrar qué proceso tenía el descriptor que impedía el fin de archivo.

Declara el uso de IA: herramienta, prompts relevantes, qué verificaste tú y qué corregiste. Lo que cuenta es que puedas explicar y comprobar cada decisión.